

Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-III) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

2. Problem Solving

7 hrs · asked in **all 41 papers** · 4 syllabus sub-topics · **375 marks, 11.1% of the archive** · **heavy solved-problem chapter**: see Ch2 – Problem Solving – Solved Problems.

Read this first:

- **One question dominates this chapter and the whole subject**: solve a crypt-arithmetic puzzle by constraint satisfaction **TOP 31/41**. **31 of the 41 papers** set it, for 6 to 8 marks, and it is almost always preceded by a 1 or 2 mark “*what is a CSP*” **TOP 12/41**. Together that is **8 to 9 marks in nearly every paper**, from one technique.
 - Thirteen different puzzles have been set. They are all worked in the companion file. **BASE + BALL = GAMES** (8 papers) and **SEND + MORE = MONEY** (6) lead.
 - **Seven of the thirteen puzzles have more than one valid answer**; **ONE + ONE + TWO = FOUR** has **107**. So the marks are in **the column-by-column constraint reasoning**, not in the digits. Write the variables, the domains, the constraints and the carries, then reason. An answer with no working scores badly even when the arithmetic is right.
 - The rest of the chapter is four small, reliable topics: **well-defined problem** **HOT 7/41**, **state space and the steps of problem solving** **HOT 5/41**, **production system** **HOT 5/41**, and the **water-jug** formulation **HOT 4/41**.
 - Pairs the examiner reliably asks together: **well-defined problem** + **a water-jug or crypt-arithmetic instance**, and “**what is a CSP**” + **the puzzle**. Almost nothing here is ever asked alone.
 - **Game playing is set up here and answered in Ch 3**. This chapter owes you only the *definition* of a game as a search problem; minimax and alpha-beta belong to §3.4.
-

2.1 Problem solving and the state space

Discuss the steps of problem solving. What is state space, successor function, goal test, path cost? 2

3

HOT 5/41 82 Bh 3, 82 Ba 2, 82 Ka 2, 81 Ch 3, 76 Bh 3 · a 2 or 3 mark opener, never more. Four steps, five terms, done

The four steps of problem solving

- **1. Goal formulation** – decide which world states count as success. Without it the agent has no way to prefer one action over another.
- **2. Problem formulation** – decide what states and actions to consider: the initial state, the successor function, the goal test and the path cost.
- **3. Searching** – find a sequence of actions leading from the initial state to a goal state, and choose the best such sequence. This is Ch 3.
- **4. Execution** – carry out the returned solution.
- **Order matters and is often the mark:** goal before formulation, and the whole search happens *before* any action is executed.

State space representation

- A **state space** is a set of **nodes**, one per state of the problem, joined by **arcs** representing the legal moves from one state to another, with a designated **initial state** and **goal state**.
- When a problem is defined with *all* its states, it is a **complete state space**.
- A state space is drawn as a **directed graph**, or a **tree** when no state repeats. Nodes are search states, arcs are the operators.

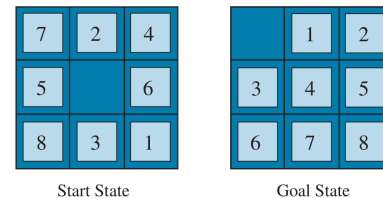
Why enumerating rules by hand does not work

- **The chess example** (*Insights* §2.1.1). A knight at (4, 4) can move to (2, 3), (2, 5), (3, 2), (3, 6), (5, 2), (5, 6), (6, 3) and (6, 5).
- Writing that out as a rule for every square gives an enormous rule set, and a chess position has around 10^{120} complete game paths.
- **The fix, and the point of the example:** describe the move as a **pattern** rather than a list, i.e. current position $\pm$ (2 vertical + 1 horizontal) or (1 vertical + 2 horizontal). One rule replaces sixty-four.
- This is exactly why the next section exists: a **production system** is a set of patterns, not a table of cases.

The five terms, with the 8-puzzle as the example

Term	Means, and for the 8-puzzle
State	A configuration of the world the agent may be in. <i>The positions of the eight tiles and the blank.</i>
Initial state	Where the agent starts. <i>The given starting board.</i>
Successor function	Given a state, returns the set of (action, resulting state) pairs reachable in one step. <i>Move the blank left, right, up or down.</i>
Goal test	Decides whether a state is a goal. <i>Does the board match the given goal board?</i>
Path cost	The sum of the step costs along a path. <i>1 per move, so the cost is the number of moves.</i>

- A **solution** is a sequence of actions from the initial state to a goal state. An **optimal solution** is one of lowest path cost.



Draw **both boards**: the initial state and the goal test are two of the five terms and neither means anything without them. The shortest solution from this pair is 26 moves.

Problem space, and how to answer “define a problem”

- **Problem space** = every state reachable, plus every action that can be taken, for a given goal. Eg the highways and cities of Nepal, when the problem is “get from Kathmandu to Dharan”.
- **The four-line answer that scores full marks** for “what are the criteria for defining a problem” (82 Ka 2):
 - define the **state space** the problem lives in,
 - specify the **initial state**,
 - specify the **goal state** or a goal test,
 - specify the **operators**, with their preconditions and effects.
- Add **path cost** if the question mentions optimality or “best”.

2.2 Well-defined problems

What makes a problem well defined? List the characteristics 2 3 4 8

HOT 7/41 71 Bh 3+4, 75 Ba 2, 80 Ash 2, 78 Ch 2, 81 Ba 2, 75 Bh 2, 81 Ash 8 · almost always a 2-mark opener to a water-jug or crypt-arithmetic question, so answer it in four bullets and move on. Only 81 Ash makes it the whole 8-mark question

Definition

- A problem is **well defined** when the agent knows, before it starts, exactly what the problem consists of. Formally, it is composed of:
 - an **initial state**,
 - the **actions**, given by a successor function,
 - a **goal test**, and
 - a **path cost** function.
- Equivalently: the starting position, the allowable operations and the goal state are all clearly specified.

Characteristics (81 Ash asks for these by name, 8)

- **Unambiguous**: one reading of what is being asked.
- **Complete**: everything needed to solve it is stated; nothing has to be assumed.
- **Verifiable**: the goal test can decide, mechanically, whether a state is a solution.
- **Finite and discrete** state space, so the states can be enumerated.
- **Deterministic operators** with known preconditions and effects.
- **Solvable in principle**: at least one action sequence reaches the goal.

Well defined Vs ill defined, with examples

	Well defined	Ill defined
Goal	Stated exactly	Vague or subjective
States	Enumerable	Open-ended
Operators	Fixed and known	Not fully known
Test	Mechanical	Needs judgement
Example	Water jug, 8-puzzle, chess, crypt-arithmetic	“Write a good poem”, “design a beautiful building”, medical diagnosis from an unclear history

- **The 5-litre / 2-litre trap** (81 Ash): the paper asks “*is this a well-defined problem? Justify.*” The answer is **yes** – the jugs, the pump, the operators and the target are all given, so every clause above is satisfied. Say that, then solve it. The companion works the solution.

2.3 Problem types and classification

Problem types and how a problem is classified 2 3

syllabus, no PYQ yet as a question of its own · but “problem types” is a printed syllabus line, and the *ignorable* / *recoverable* / *irrecoverable* split is worth knowing because it explains why the water jug is easy and tic-tac-toe is not

By whether a move can be undone (*Insights* §2.2.1)

- **Ignorable** – intermediate steps can simply be ignored; a wrong move costs nothing. *Eg* theorem proving, the **water jug**.
 - Solvable with a simple control strategy, no backtracking needed.
- **Recoverable** – steps can be undone to get back to an earlier state. *Eg* the **8-puzzle**.
 - Needs backtracking, so the control strategy must keep a history.
- **Irrecoverable** – steps cannot be undone. *Eg* **tic-tac-toe**, chess.
 - Needs **planning**: consider the consequences before moving, because there is no second chance. This is why games need adversarial search (Ch 3).
- **Decomposable** – the problem breaks into independent sub-problems whose answers combine. *Eg* symbolic integration, word puzzles.

By what the agent can see and predict

- **Single-state problem** – deterministic and fully accessible. The agent always knows exactly which state it is in. *Eg* chess with the board in view.
- **Multiple-state problem** – deterministic but inaccessible. The agent knows the rules but not its exact state, so it reasons over a *set* of possible states. *Eg* walking across a dark room. It may have no sensors at all.
- **Contingency problem** – non-deterministic and inaccessible. The environment is partially observable or the actions are uncertain, so the agent **must sense during execution** and cannot plan the whole path in advance.
- **Exploration problem** – the state space itself is unknown. The agent discovers states and operators as it acts. *Eg* a robot in an unmapped maze. **The hardest of the four**, and the one that needs learning.

2.4 Production systems

Define a production system. What are its essential terms? Explain its role with an example 2 4 6

HOT 5/41 70 Ma 2+6, 71 Bh 4, 82 Ka 2, 77 Ch 2, 75 Bh 2+6 · 70 Ma pairs it with *why is searching necessary in AI*; 75 Bh pairs it with *well-defined problems*. It is the machinery behind the water-jug rules, so answer it with those rules as the example

Definition and the four components

- A **production system** is a set of **rules**, each of the form $C_i \rightarrow A_i$:
 - **LHS** C_i , the **pattern** or condition, which decides whether the rule applies;
 - **RHS** A_i , the **action** to perform if it is applied.
- The right-hand side is read as an **action recommendation**, not as a logical conclusion. That is what separates a production rule from an implication.
- **The four components**, and the answer to “essential terms”:
 - **Rule base** (production rules / rule memory) – the rules themselves.
 - **Working memory** (database) – the current state of the world, updated as rules fire.
 - **Control strategy** (inference engine) – decides which applicable rule to fire when several match, and when to stop.
 - **Rule applier** – carries out the chosen action.
- **The recognise-act cycle**: **match** the rules against working memory → **conflict resolution** picks one from the matching set → **act**, i.e. fire it and update working memory → repeat until the goal test passes or nothing matches.

Two requirements of a good control strategy

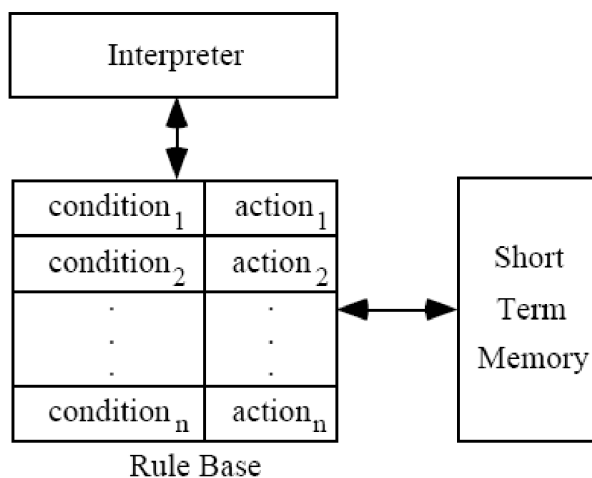
- **It must cause motion.** A strategy that can pick the same rule forever never leaves the initial state.
- **It must be systematic.** Choosing at random causes motion but wanders, and may revisit states endlessly.
- Both are one-line answers and both are asked.

Example: the vacuum world

Pattern (LHS)	Action (RHS)
[A, clean]	move Right
[A, dirty]	Suck
[B, clean]	move Left
[B, dirty]	Suck

- **The example that earns the marks** is the **water jug**, whose eight production rules are in the companion. A question asking for a production system with an example almost always means that one.
- **Practical uses**: expert systems (Ch 7), autonomous robots, and cognitive architectures modelling human reasoning such as ACT and SOAR.

The architecture, which is what to draw



- **The same four components under other names.** This drawing says **Interpreter** for the control strategy and inference engine, **Short Term Memory** for working memory, and folds the rule applier into the interpreter. Either set of names is accepted; **use one set consistently** and say which.
- Read the two arrows as the **recognise-act cycle**. The interpreter matches the rule base against short term memory, resolves the conflict set down to one rule, and fires it; firing rewrites short term memory, and the cycle repeats.
- **Both arrows are double-headed**, which is the part worth noticing: the rules read the state and the state is rewritten by the rules. Nothing else in the system holds any information, so **the whole state of the computation is the contents of that one box**.

Problems solvable by production rules 6

75 Bh 2+6 · asked as “explain about problems that can be solved using production rules with an example”

Which problems suit a production system

- The state can be written down **compactly** – the water jug’s whole state is the pair (x, y) .
- The legal moves are **few, uniform and pattern-like**, so a handful of rules covers every case.
- Progress is made by **repeatedly rewriting the state**, not by evaluating a formula.
- The goal is a **recognisable pattern**, so the goal test is a match.

The standard examples, and what each shows

- **Water jug** – 8 rules, state (x, y) . The canonical answer.
- **8-puzzle** – 4 rules (move the blank up, down, left, right), state = the board. Shows a recoverable problem needing backtracking.
- **Missionaries and cannibals, farmer/wolf/goat/cabbage** – rules with a **safety precondition**, which is what makes them interesting.
- **Tower of Hanoi** – shows a decomposable problem.
- **Tic-tac-toe** – shows an irrecoverable one, where rules alone are not enough and search over an opponent is needed.

2.5 Constraint satisfaction problems

What is a Constraint Satisfaction Problem? Discuss it and list its constraints 1 2 3

TOP 12/41 81 Bh, 80 Bh, 80 Ba, 78 Ba, 80 Ch, 74 Bh 2, 79 Jth 1, 79 Ch 3, 74 Ma, 72 Ash, 78 Po, 70 Bh · **never asked alone**: it is the 1–3 mark preamble to a crypt-arithmetic puzzle in every one of these papers, so keep it short and get to the puzzle

Definition

- A **constraint satisfaction problem** is a search procedure that **operates in a space of constraints** rather than a space of states.
- States are defined by the **values assigned to variables**, and the goal test is a set of **constraints those values must obey**.
- Formally, a CSP is the triple:
 - **Variables** $X = \{X_1, X_2, \dots, X_n\}$, the unknowns to be assigned.
 - **Domains** $D = \{D_1, D_2, \dots, D_n\}$, the set of values each variable may take.
 - **Constraints** $C = \{C_1, C_2, \dots, C_m\}$, the restrictions on which combinations of values are allowed.
- **Solution**: an assignment of one domain value to every variable that violates no constraint. **Write these three lines first in any crypt-arithmetic answer** – they are worth marks on their own.

How it is solved: constraint propagation

- Constraints are **discovered and propagated as far as possible** through the system. Where propagation stalls, a **guess** is made and added as a new constraint, and propagation resumes.
- If the guess leads to a contradiction, **backtrack** and guess otherwise.

The two termination conditions

- A **contradiction is detected** – no solution is consistent with the constraints now known.
- **Propagation has run out of steam** – no further change can be made from current knowledge. Either a solution has been found, or a guess must be made.
- Both are asked as a pair. Two lines, two marks.

Types of constraint

- **Unary** – restricts one variable. *Eg* a leading letter is not 0.
- **Binary** – relates two. *Eg* two regions differ in colour.
- **Higher-order** – relates three or more. *Eg* a crypt-arithmetic column, $D + E = 10C_1 + Y$.
- **Global / AllDiff** – all variables take different values. Every crypt-arithmetic puzzle carries this one.

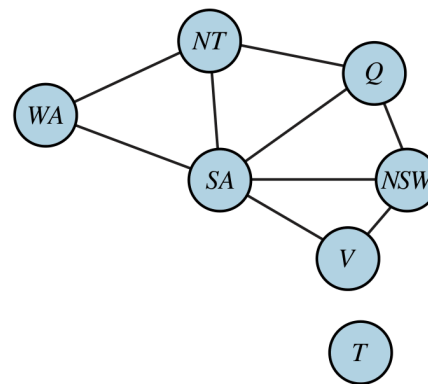
Other standard CSP examples

- **Map colouring** – no two adjacent regions share a colour.
- **n -queens** – no queen attacks another.
- **Class scheduling, VLSI layout, Sudoku**.

Map colouring, and the same problem as a constraint graph



(a)



(b)

(a) the seven regions, (b) the CSP. Variables become **nodes**, binary constraints become **edges**; here every edge means “these two must differ”. The domain of each variable is {red, green, blue}. · **Two things to read off (b) that (a) hides**: *SA* touches five of the other six, so it is the variable to assign first and the one a most-constrained-variable heuristic picks; and *T* has **no edge at all**, so Tasmania is an independent sub-problem that can be coloured any colour whatever the rest does. · That is the whole point of drawing the graph: the structure of the problem is visible in it and is invisible in the map.

The crypt-arithmetic rules, and how to set the answer out 6 7 8

TOP 31/41 · the highest-value single technique in the subject · every puzzle is worked in Ch2 – Problem Solving – Solved Problems

The rules, as every paper states them

- Different letters denote different digits, and identical letters denote the same digit. This is the AllDiff constraint.
- No leading letter may be 0.
- Digits come from $D = \{0, 1, 2, \dots, 9\}$, so a puzzle with more than **10 distinct letters has no solution at all**.
- Any extra restriction the question prints overrides these.

The five-step method

(Insights §2.3)

- **1.** Write the sum in columns with a **carry variable** $C_1, C_2, \dots$ above each column boundary.
- **2.** List X , the set of variables.
- **3.** List D , the domain.
- **4.** List C : the leading-letter constraints, the AllDiff constraint, and **one equation per column**, $(\text{column sum}) + C_{\text{in}} = 10 C_{\text{out}} + (\text{result digit})$.
- **5.** Reason column by column, redrawing the sum with the digits filled in after each deduction, until every letter is fixed. Close with the arithmetic check.

The three deductions that crack most puzzles

- **A carry out of the leftmost column can only be 1.** If the answer is one digit longer than the addends, that leading digit is 1. This is what gives $M = 1$ in $\text{SEND} + \text{MORE} = \text{MONEY}$ immediately.
- **A letter added to itself.** $2N$ ends in the same letter only when that letter is 0 or 5, which is how $\text{TEN} + \text{TEN} + \text{FORTY} = \text{SIXTY}$ is cracked.
- **A column where a letter repeats.** If $E + O + C_2 = N$ and also $N + R + C_1 = E + 10$, substitute one into the other; the unknowns cancel.

Two things that lose marks

- **Giving only the digits.** Most of these puzzles have many solutions, so an unjustified assignment cannot be marked. Show the columns.
- **Forgetting the AllDiff check at the end.** It is the single commonest way a nearly-right answer fails, and it is what rules out $B = 9$ in the $AB + CD = AAA$ question (76 Bh).

2.6 Game playing

Formulate a game as a search problem 2 7

PIN 1/41 72 Ma 7, tic-tac-toe state space · **the algorithms live in §3.4**, not here. This section only owes you the formulation; minimax and alpha-beta are Ch 3, where they are asked 14 times

A game, defined as a search problem

- **Initial state** – the board as set up, plus whose turn it is.
- **Operators** – the legal moves.
- **Terminal test** – says when the game is over.
- **Utility / payoff function** – a number on each terminal state saying who won and by how much. Eg +1 win, 0 draw, -1 loss.
- **What makes it different from ordinary search:** there is an **opponent**, choosing to make things as bad as possible for you. So a plan cannot be a fixed sequence of moves; it must be a **strategy**, an answer for every reply.
- Two-player, perfect information, zero-sum games are solved by **minimax** (§3.4), made efficient by **alpha-beta pruning**.

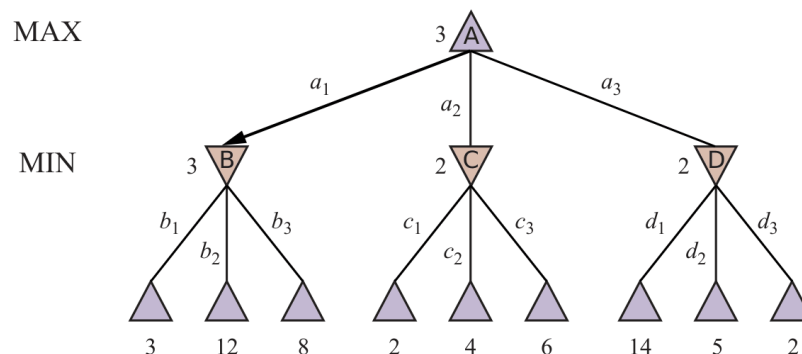
Why games were chosen as an AI testbed

- Rules are **simple and unambiguous**, so no knowledge acquisition problem gets in the way.
- The state is **fully observable**, so no perception problem either.
- Yet the **search space is enormous**, so brute force fails and real technique is needed. That combination is rare and useful.
- Performance is **measurable** against human players.

Production systems Vs game playing

- A production system fires rules against **its own** working memory.
- A game must additionally model **another agent choosing against it**, which no rule set alone can express. That is precisely why Ch 3 needs a separate family of algorithms.

The formulation above, drawn



MAX moves from the root, MIN replies, and the nine leaves carry the **utility**. The number written on each internal node is backed up from its children, and the arrow marks MAX's choice. · **The tree is the strategy**, which is the point of the whole section: MAX cannot commit to a sequence, only to an answer at every node MIN might send it to. · Computing those backed-up numbers is minimax, and it is §3.5, not here.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.